Warsztat

Analiza opinii klientów w SQL Server z modelem Hugging Face

Szkolenie „Wykorzystanie AI w pracy z SQL Server”

Parametr	Wartość
Poziom	Zaawansowany T-SQL, podstawy Pythona; SQL Server 2025 Developer, SSMS
Model	`distilbert-base-uncased-finetuned-sst-2-english` (klasyfikacja sentymentu, lokalnie)
Technologie	SQL Server, Machine Learning Services, `sp_execute_external_script`, Python (transformers, torch, pandas)
Efekt końcowy	Działający w bazie pipeline: opinie → model → wyniki → analiza biznesowa i alerty

Spis treści

Wprowadzenie 3
Scenariusz biznesowy 3
Wymagania wstępne 3
Moduł 1. Środowisko i dane 4
Krok 1.1. Schemat i reset środowiska 4
Krok 1.2. Tabele 4
Krok 1.3. Dane testowe 4
Krok 1.4. Test modelu (smoke test) 4
Moduł 2. Procedura analizy sentymentu 5
Krok 2.1. Tabela wyników 5
Krok 2.2. Procedura usp_ScoreReviews 5
Krok 2.3. Uruchomienie w trzech przebiegach 5
Moduł 3. Analiza biznesowa i alerty 7
Krok 3.1. Widok łączący opinie z wynikami 7
Krok 3.2. Ranking produktów 7
Krok 3.3. Alerty: rozbieżność gwiazdek i treści 7
Krok 3.4. Automatyczna analiza nowej opinii 7
Zadania dodatkowe 9
Rozwiązywanie problemów 9

Wprowadzenie

Scenariusz biznesowy

Pracujesz jako analityk danych w sklepie internetowym **TechShop**, który sprzedaje elektronikę i akcesoria komputerowe. Klienci zostawiają opinie z oceną od 1 do 5 gwiazdek. Dział obsługi klienta ma dwa problemy:

- nie wie, które produkty są **najgorzej odbierane** na podstawie treści opinii, a nie tylko średniej gwiazdek,
- część klientów **klika złą liczbę gwiazdek**: pisze skargę i daje 5, albo chwali produkt i daje 1. Takie opinie zaburzają statystyki i często oznaczają niezałatwioną reklamację.

Twoim zadaniem jest zbudowanie rozwiązania, które **w całości działa wewnątrz SQL Server**: pobiera opinie z tabeli, przepuszcza je przez model Hugging Face, zapisuje wyniki i udostępnia widoki z analizą oraz listą opinii wymagających interwencji.

Wymagania wstępne

- SQL Server z zainstalowanym **Machine Learning Services** i włączonym `external scripts enabled`.
- Zarejestrowany język zewnętrzny **Python** w bazie, w której pracujesz (tak jak na poprzednich laboratoriach).
- Model **distilbert** pobrany do `C:\ProgramData\HF_MODELS\distilbert` (katalog widoczny dla AppContainer).
- Pakiety Pythona: `transformers`, `torch`, `pandas`.

> **Czym ten warsztat różni się od laboratorium?**
>
> Na laboratorium teksty trafiały do procedury jako jeden ciąg rozdzielony znakiem `|`, a wynik nie był powiązany z żadnym rekordem. Teraz pracujemy jak w produkcji:
>
> - dane wejściowe pochodzą **z tabeli**, a wynik łączy się z opinią przez `review_id`,
> - procedura przetwarza **tylko nowe** opinie, więc można ją uruchamiać wielokrotnie,
> - model przetwarza **całą paczkę naraz**, zamiast pętli po pojedynczych tekstach,
> - wyniki modelu **porównujemy z danymi biznesowymi** (gwiazdkami).

> **Uwaga o modelu**
>
> Model `distilbert-...-sst-2-english` został wytrenowany na tekstach angielskich, dlatego opinie w danych testowych są po angielsku. Polskie teksty model sklasyfikuje, ale wyniki będą przypadkowe. Wariant z modelem wielojęzycznym znajdziesz w zadaniach dodatkowych.

Moduł 1. Środowisko i dane

Cel: przygotować od zera schemat i dane testowe oraz potwierdzić, że SQL Server potrafi uruchomić model Hugging Face.

Krok 1.1. Schemat i reset środowiska

Skrypt usuwa wszystkie obiekty warsztatu, jeśli już istnieją. Dzięki temu w razie problemów możesz zacząć od nowa, uruchamiając go jeszcze raz.

Skrypt: m1_basic.sql

Krok 1.2. Tabele

Skrypt: m1_table.sql

Krok 1.3. Dane testowe

Dane zawierają 8 produktów i 40 opinii (po 5 na produkt). Komentarz `/* 01 */` na początku wiersza to numer `review_id`, jaki otrzyma opinia. Większość ocen zgadza się z treścią, ale **4 opinie celowo mają rozbieżność** między gwiazdkami a tekstem. Wykryjesz je w module 3.

Skrypt: m1_data.sql

Krok 1.4. Test modelu (smoke test)

Zanim zbudujesz procedurę, sprawdź, czy Python w SQL Server ładuje model i zwraca poprawny wynik dla jednego zdania. Pierwsze uruchomienie może potrwać kilkanaście sekund, bo model jest wczytywany z dysku.

Skrypt: m1_test_model.sql

WERYFIKACJA

A. Wynik smoke testu: jeden wiersz z `label = POSITIVE` i `score` powyżej `0.99`.

B. Uruchom zapytanie kontrolne. Wszystkie testy muszą zwrócić `OK`:

Skrypt: m1_verify.sql

Jeśli test 5 zwraca BŁĄD, pracujesz w innej bazie niż ta, w której zarejestrowano `Python`. Przełącz bazę (`USE ...`) i zacznij od kroku 1.1.

Moduł 2. Procedura analizy sentymentu

Cel: zbudować procedurę, która analizuje opinie bez wyniku, zapisuje wynik powiązany z `review_id` i może być bezpiecznie uruchamiana wielokrotnie (np. co godzinę z SQL Server Agent).

Krok 2.1. Tabela wyników

Zwróć uwagę na ograniczenia: `PRIMARY KEY` na `review_id` gwarantuje, że opinia nie zostanie przeanalizowana dwa razy. `CHECK` pilnuje poprawnych etykiet i zakresu pewności. Kolumna `model_name` zapisuje, którym modelem uzyskano wynik, co przyda się po zmianie modelu.

Skrypt: m2_table.sql

Krok 2.2. Procedura usp_ScoreReviews

Procedura działa w czterech krokach: wybiera opinie bez wyniku, sprawdza, czy jest co analizować, uruchamia model na całej paczce i zapisuje wyniki.

Skrypt: m2_procedure.sql

Na co zwrócić uwagę w kodzie

- `NOT EXISTS` w kroku 1 odpowiada za przetwarzanie przyrostowe: opinie z wynikiem są pomijane.
- `IF NOT EXISTS ... RETURN` chroni przed uruchomieniem Pythona na pustym zbiorze. Tokenizer zgłosiłby wtedy błąd, a ładowanie modelu i tak trwa kilka sekund.
- `padding=True` pozwala przetworzyć wszystkie teksty jednym wywołaniem modelu, zamiast pętli `for` z laboratorium.
- `@batch_size` ogranicza liczbę opinii w jednym przebiegu. Przy tysiącach opinii chroni to przed zużyciem całej pamięci.
- W skrypcie Pythona używamy **cudzysłowów podwójnych**, bo cały skrypt jest literałem T-SQL w apostrofach.

Krok 2.3. Uruchomienie w trzech przebiegach

Uruchom procedurę trzy razy i obserwuj zakładkę **Messages** oraz liczbę wierszy w tabeli wyników.

Skrypt: m2_run.sql

WERYFIKACJA

A. Przebiegi: po pierwszym przebiegu tabela ma 10 wierszy, po drugim 40, a trzeci wyświetla „Brak nowych opinii do analizy.” i nie dodaje nic.

B. Uruchom zapytanie kontrolne. Wszystkie testy muszą zwrócić `OK`:

Skrypt: m2_verify.sql

Jeśli procedura zgłasza błąd naruszenia klucza głównego, warunek `NOT EXISTS` jest napisany niepoprawnie i opinie są analizowane ponownie.

Moduł 3. Analiza biznesowa i alerty

Cel: zamienić surowe wyniki modelu w informacje dla działu obsługi klienta: ranking produktów, listę podejrzanych opinii i automatyczną analizę nowo dodanych opinii.

Krok 3.1. Widok łączący opinie z wynikami

Gwiazdki zamieniamy na sentyment „oczekiwany" (4–5 = pozytywny, 1–2 = negatywny, 3 = neutralny, pomijany) i porównujemy go z wynikiem modelu.

Skrypt: m3_view.sql

Krok 3.2. Ranking produktów

Skrypt: m3_rank.sql

Porównaj kolumny `avg_rating` i `negative_pct`. Czy produkt z najniższą średnią gwiazdek jest też tym z najwyższym odsetkiem negatywnych treści?

Krok 3.3. Alerty: rozbieżność gwiazdek i treści

Alert powstaje, gdy model ocenia treść inaczej niż wynika z gwiazdek i jest przy tym pewny (score ≥ 0.80). Próg ogranicza liczbę fałszywych alarmów.

Skrypt: m3_view2.sql

Krok 3.4. Automatyczna analiza nowej opinii

Procedura `usp_AddReview` dodaje opinię i od razu wywołuje `usp_ScoreReviews`. Dzięki przetwarzaniu przyrostowemu z modułu 2 model analizuje tylko tę jedną nową opinię

Skrypt: m3_addreview.sql

WERYFIKACJA

A. Wynik procedury usp_AddReview: jeden wiersz z `rating = 5`, `sentiment = NEGATIVE` i `is_match = 0`.

B. Uruchom zapytanie kontrolne. Wszystkie testy muszą zwrócić `OK`:

Skrypt: m3_verify.sql

> **Jeśli test 3 zwraca BŁĄD**, sprawdź, czy dane z kroku 1.3 zostały wstawione do pustej tabeli. `review_id` jest kolumną `IDENTITY`, więc po wcześniejszym, nieudanym `INSERT` numeracja się przesuwa. Uruchom moduł 1 od kroku 1.1.

Zadania dodatkowe

Dla osób, które skończą wcześniej:

1. **Próg pewności.** Dodaj do `vw_ReviewAlerts` parametryzację przez funkcję tabelaryczną `sklep.fn_ReviewAlerts(@min_score FLOAT)` i sprawdź, ile alertów zostaje przy progach 0.6, 0.8 i 0.95.
2. **Model wielojęzyczny.** Pobierz model `nlptown/bert-base-multilingual-uncased-sentiment` (zwraca 1–5 gwiazdek), dodaj polskie opinie i porównaj przewidywaną liczbę gwiazdek z oceną klienta.
3. **Ponowna analiza.** Napisz procedurę `sklep.usp_RescoreProduct @product_id`, która usuwa wyniki dla jednego produktu i analizuje je ponownie.
4. **Harmonogram.** Utwórz zadanie SQL Server Agent uruchamiające `usp_ScoreReviews` co 15 minut.

Rozwiązywanie problemów

Objaw	Przyczyna i rozwiązanie
`Language Python does not exist`	Język jest zarejestrowany w innej bazie. Przełącz bazę albo wykonaj `CREATE EXTERNAL LANGUAGE` w bieżącej.
`OSError: ... distilbert ... not found`	Zła ścieżka do modelu lub brak uprawnień AppContainer do katalogu. Sprawdź `C:\ProgramData\HF_MODELS\distilbert` i uprawnienia `ALL APPLICATION PACKAGES`.
Próba pobierania z internetu, timeout	Brak `HF_HUB_OFFLINE=1` lub `local_files_only=True`. Proces SQL Server nie ma dostępu do sieci.
`Column name or number of supplied values does not match`	Kolumny w `OutputDataSet` nie zgadzają się z listą kolumn w `INSERT`. Python musi zwrócić dokładnie: `review_id`, `sentiment`, `score`.
`Violation of PRIMARY KEY constraint`	Opinia jest analizowana drugi raz. Sprawdź warunek `NOT EXISTS` w procedurze.
`SyntaxError` w Pythonie	W skrypcie użyto apostrofu, który zamknął literał T-SQL. W kodzie Pythona używaj cudzysłowów podwójnych.
Weryfikacja z konkretnymi `review_id` nie przechodzi	Numeracja `IDENTITY` przesunęła się po nieudanym `INSERT`. Uruchom moduł 1 od kroku 1.1.